Programming Assignment II: Summary

CS683: Advanced Computer Architecture, Autumn 2026
Computer Science and Engineering
Indian Institute of Technology, Bombay

Team Member	Roll No
Tejeswar	24b0918
Bhavesh Ramakrishnan Karthik	24b0945
Sushant Padha	24b1057
Rhythm Kedia	24b1071

Contents

Introduction **3**

Implementation files . . . 3

Methodology . . . 3

Task 1: Stride Pattern Prefetching **4**

Task 1A: IP-stride prefetcher . . . 4

Task 1B: Design . . . 4

Storage overhead . . . 4

Accuracy throttling . . . 4

Results . . . 4

Task 2A: Instruction-Conscious L2 Replacement Policy (I-RRIP) **5**

Interface change . . . 5

Design: insertion, promotion, eviction . . . 5

Results . . . 8

Task 2B: Dynamic I-RRIP **10**

Leader sets and decision logic 10

Sensitivity study 10

Results 10

Task 3: Incorporating SPP with Dynamic I-RRIP **11**

Handling prefetched lines 11

Results 11

Introduction

Report from team flipflops.

Implementation files

Our code is on the `dev` branch of the repository.

```
prefetcher/spp.l1d_pref             Task 1B: Stride Pattern Prefetcher
replacement/irrip.l2c_repl          Task 2A: I-RRIP
replacement/dyn_irrip.l2c_repl      Tasks 2B and 3: Dynamic I-RRIP
```

Methodology

- Every provided trace is simulated with 25M warmup and 25M simulation instructions.
- The baseline has no L1D prefetcher and LRU replacement at L2C. Speedup is IPC divided by the baseline's IPC on the same trace.

Task 1

Stride Pattern Prefetching

Task 1A: IP-stride prefetcher

Task 1B: Design

Storage overhead

Accuracy throttling

Results

Task 2A

Instruction-Conscious L2 Replacement Policy (I-RRIP)

Interface change

We added a last parameter, `uint8_t isInstruction`, to `update_replacement_state`: in the member-function pointer type in `inc/cache.h`, in every cache level's definition, and at every call site. Each call in `src/cache.cc` passes the `instruction` field of the packet it handles: the MSHR entry on a fill from the lower level, the write queue entry on a writeback that misses, and the read, write or prefetch queue entry on a hit. The BTB calls in `src/ooo_cpu.cc` pass 0. The flag is also set on fills of page-table lines for instruction translations, so I-RRIP protects those lines too; the L2C instruction load MPKI does not count them, because it counts only load requests. I-RRIP stores the flag of each line when the line is filled, because `find_victim` does not receive it.

Design: insertion, promotion, eviction

An L1I miss stalls fetch; the out-of-order window can only keep executing instructions already fetched, so it hides little of the miss. A data miss is often overlapped with independent work in the window. An L2 hit is therefore worth more for an instruction line than for a data line, and I-RRIP gives up some data hits to keep instruction lines. It starts from SRRIP with 2-bit RRPVs (maximum RRPV 3), which inserts every line at RRPV 2, promotes every hit to RRPV 0, and evicts the first line at RRPV 3, aging all lines until one reaches 3.

	SRRIP	I-RRIP (final, level 8)
Insertion	every line at 2	instruction lines at 2 (unchanged); data lines at 3
Promotion	every hit to 0	instruction hit to 0 (unchanged); data hit: RRPV − 1
Eviction	first line at 3; else age all lines	first *data* line at 3, else any line at 3; else age: data lines every round, instruction lines every 4th round

Table 1: Replacement rules of SRRIP and the final I-RRIP.

What the eviction rule does. A data line needs at most three aging rounds to reach RRPV 3, and an instruction line is aged only in the fourth round. So in a set that holds at least one data line, the victim search ends before any instruction line is aged. An instruction line is then evicted only if it already has RRPV 3, which it can reach only by aging while its set holds no data line. **TODO: how often a set that holds a data line still evicts an instruction line is not measured.**

How we chose the rules. `irrip.l2c_repl` implements eight variants, which we call levels; the default is level 8, the final policy. Each level changes one rule of a previous one:

- Level 1: data-first eviction, instruction lines aged every 2nd round, every line inserted at 2, instruction hit to 0, data hit RRPV − 1.

- Level 2: level 1, but instruction lines inserted at 0.
- Level 3: level 1, plus a cap of 6 instruction lines per 8-way set: when a set holds 6 or more, the instruction line with the highest RRPV is evicted.
- Level 4: level 2, but data hits promoted to 1. Level 5: level 1, but data hits promoted to 1.
- Level 6: level 1, but instruction lines aged every 4th round.
- Level 7: level 1, but data lines inserted at 3.
- Level 8: levels 6 and 7 together.

		Load MPKI			MPKI ratio to SRRIP		
Policy	Speedup	L2C	Instr.	Data	L2C	Instr.	Data
SRRIP	1.0004	15.81	9.85	5.96	1.000	1.000	1.000
level 1	1.0182	13.55	6.35	7.20	0.887	0.656	1.173
level 2	1.0089	14.47	7.08	7.39	0.934	0.719	1.199
level 3	1.0329	14.99	8.20	6.79	0.979	0.881	1.125
level 4	1.0084	14.50	7.12	7.38	0.936	0.725	1.197
level 5	1.0178	13.56	6.37	7.19	0.888	0.660	1.172
level 6	1.0194	13.85	6.15	7.70	0.904	0.627	1.240
level 7	1.0216	13.73	5.81	7.92	0.901	0.597	1.277
level 8 (I-RRIP)	1.0216	13.91	5.76	8.14	0.911	0.590	1.306

Table 2: Every level, averaged over the three traces. Speedup is over LRU (geometric mean). MPKI columns are arithmetic means; ratio columns are each MPKI relative to SRRIP (geometric mean of the per-trace ratios). Percentages in the text compare the MPKI columns.

Insertion.

- *Instruction lines: not modified (RRPV 2).* Inserting them at 0 (level 2 vs. level 1) raises mean instruction MPKI by 11% and mean data MPKI by 3%, and lowers the speedup from 1.0182 to 1.0089. At RRPV 2 an instruction line needs one L2 hit to reach RRPV 0, so only instruction lines that are reused at the L2 get full protection. At RRPV 0, every instruction fill gets full protection, including lines that are not reused before they are evicted, and data-first eviction keeps them for a long time. **TODO: we have not measured how many instruction fills are never reused at the L2; this explanation is consistent with both MPKIs rising, but not proven.**
- *Data lines: modified (RRPV 3).* Level 7 vs. level 1 lowers instruction MPKI by 8.6% and raises data MPKI by 10%, and the speedup rises from 1.0182 to 1.0216. A data line inserted at 3 is a victim candidate ahead of every instruction line unless it is hit first, so data that is not reused soon cannot push instruction lines out. A hit moves it to RRPV 2; it is still evicted before any instruction line once it ages back to 3.

Promotion.

- *Instruction hits: not modified (RRPV 0).* A hit is the only evidence of reuse, and RRPV 0 gives a reused instruction line the most protection. We did not test other values. **TODO: no ablation of instruction promotion.**

- *Data hits: modified (RRPV − 1).* The data promotion rule barely matters: promoting data hits to 1 instead (level 5 vs. level 1, level 4 vs. level 2) changes the mean speedup by at most 0.05% and each mean MPKI by at most 0.05. **TODO: why the data promotion rule does not matter is not verified.**

Eviction.

- *Modified: data first, instruction lines aged more slowly.* Aging instruction lines every 4th round instead of every 2nd (level 6 vs. level 1) lowers instruction MPKI by 3.1% and raises data MPKI by 6.9%, for a speedup of 1.0194 instead of 1.0182. **TODO: we did not measure data-first eviction alone against SRRIP; its effect is measured only together with the other rules (level 1).**
- On top of data insertion at 3, slower aging adds little (level 8 vs. level 7): instruction MPKI falls by 0.8%, data MPKI rises by 2.8%, and the speedup is the same to four digits (1.0216). **TODO: how often the victim search reaches the aging step is not measured; the 2.8% data MPKI rise shows it is reached often enough to matter.** We keep level 8 because it has the lowest instruction MPKI, which is the goal of this task.

The instruction cap (level 3). Level 3 has the highest speedup (1.0329), but its instruction MPKI is 29% above level 1's, so it does not meet the goal of this task, and we did not choose it. The cap lowers LLC load misses (Table 3): on Trace1, from 301.9k with level 1 to 267.4k, while L2C load misses rise from 447.4k to 464.5k. **TODO: why the cap lowers LLC misses is not verified. Our hypothesis: the cap sends instruction lines back to the LLC more often, which keeps them recent in the LRU LLC. ChampSim does not split LLC statistics into instruction and data, so we cannot test it from the current logs. TODO: the cap value (6) comes from a sweep over 4 to 8 at shorter, non-final instruction counts; rerun it at the final settings or drop the claim.**

Trace	Policy	L2C load misses	LLC load hits	LLC load misses
Trace1	LRU	442.4	135.5	306.9
	level 1	447.4	145.5	301.9
	level 3	464.5	197.1	267.4
	level 8	465.0	166.2	298.8
Trace2	LRU	683.9	478.7	205.3
	level 1	474.8	266.0	208.8
	level 3	555.7	367.2	188.5
	level 8	481.2	275.7	205.5
Trace3	LRU	98.9	20.3	78.6
	level 1	94.2	17.9	76.3
	level 3	104.2	30.4	73.9
	level 8	96.7	21.7	75.0

Table 3: L2C load misses and LLC load hits and misses, in thousands, over the 25M simulated instructions.

Results

Trace	Policy	IPC	Speedup	L2C	L2C instr.	L2C data
Trace1	LRU (baseline)	0.2323	1.0000	17.70	10.34	7.36
	SRRIP	0.2322	0.9995	17.64	10.56	7.08
	I-RRIP	0.2339	1.0068	18.60	8.98	9.62
Trace2	LRU (baseline)	0.2512	1.0000	27.36	18.85	8.50
	SRRIP	0.2518	1.0026	25.75	17.78	7.96
	I-RRIP	0.2639	1.0509	19.25	7.63	11.62
Trace3	LRU (baseline)	0.3821	1.0000	3.95	1.12	2.83
	SRRIP	0.3817	0.9991	4.04	1.20	2.84
	I-RRIP	0.3850	1.0076	3.87	0.67	3.19
Mean	LRU (baseline)	0.2885	1.0000	16.34	10.11	6.23
	SRRIP	0.2886	1.0004	15.81	9.85	5.96
	I-RRIP	0.2943	1.0216	13.91	5.76	8.14

Table 4: IPC, speedup over LRU, and L2C load MPKI (total, instruction, data). L2C load MPKI is the sum of instruction and data load MPKI. Mean: geometric for speedup, arithmetic otherwise.

		Load MPKI			MPKI ratio to LRU		
Policy	Speedup	L2C	Instr.	Data	L2C	Instr.	Data
LRU (baseline)	1.0000	16.34	10.11	6.23	1.000	1.000	1.000
SRRIP	1.0004	15.81	9.85	5.96	0.986	1.009	0.967
I-RRIP	1.0216	13.91	5.76	8.14	0.898	0.595	1.263

Table 5: Means over the three traces (geometric for speedup, arithmetic for MPKI), and each MPKI relative to LRU (geometric mean of the per-trace ratios).

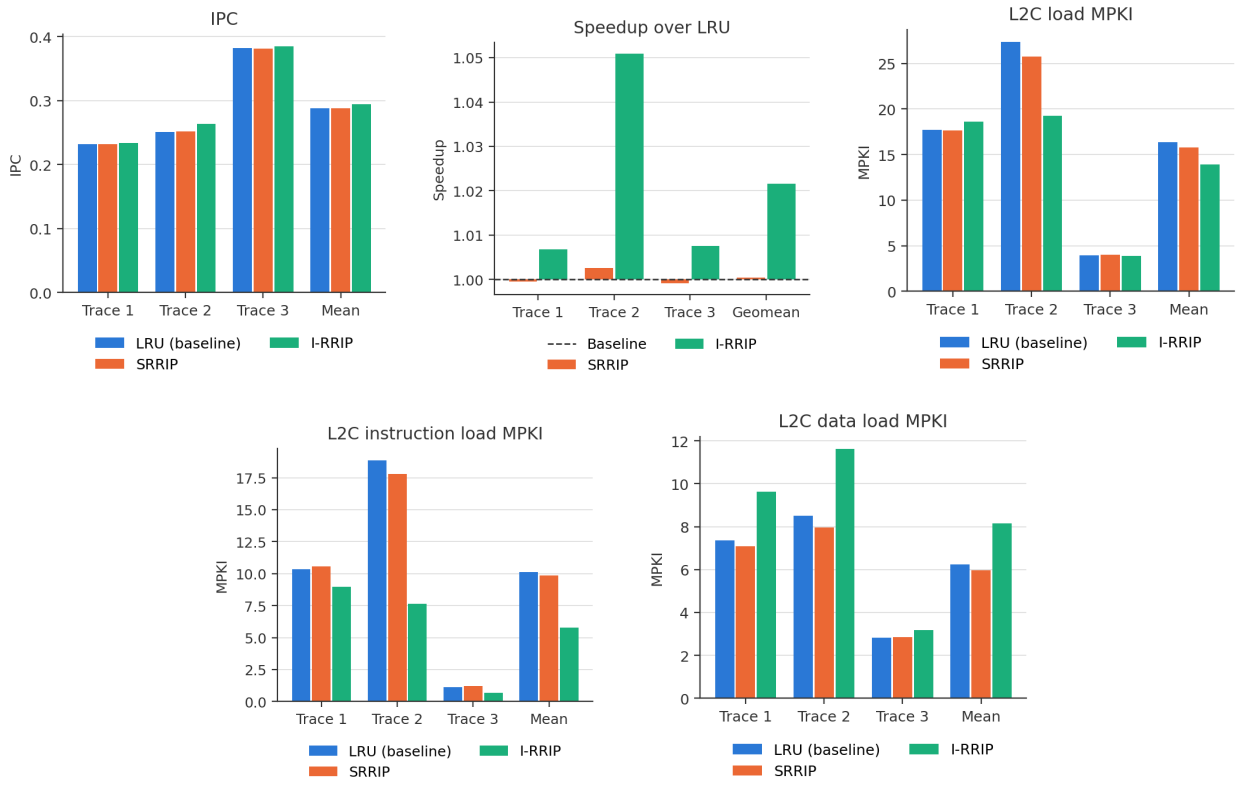


Figure 1: IPC, speedup over LRU (dashed line), and L2C load MPKI per trace.

I-RRIP lowers L2C instruction load MPKI on every trace: by 13% on Trace1, 60% on Trace2 and 40% on Trace3. Data load MPKI rises on every trace, by 31%, 37% and 13%. SRRIP alone is within 0.3% of LRU on every trace, so the gain comes from the instruction priority, not from RRIP.

Instruction misses cost more than data misses. On Trace1, I-RRIP has 5% *more* L2C load misses than LRU (18.60 vs. 17.70 MPKI), yet its IPC is 0.7% higher: it trades 1.35 MPKI of instruction misses for 2.26 MPKI of data misses. This is consistent with the argument above that an instruction miss stalls fetch while a data miss is partly overlapped.

Trace2 gains the most. The L1I MPKI, which the L2 policy does not change, is 14.5 on Trace1, 40.0 on Trace2 and 2.1 on Trace3. Trace2 sends the most instruction misses to the L2 and gains the most (5.1%). Trace1 and Trace3 gain about the same (0.7% and 0.8%) despite a 7× difference in L1I MPKI, so L1I MPKI alone does not predict the gain.

Why a 60% cut gives 5%. On Trace2, I-RRIP removes 202.7k L2C load misses relative to LRU. LLC load hits fall by 203.0k over the same run, and LLC load misses stay at 205.3k vs. 205.5k (Table 3). So the misses that I-RRIP removes were LLC hits, not DRAM accesses. Each one saves the difference between an LLC hit and an L2 hit (20 cycles: the LLC latency on top of the L2C latency in this configuration), not a DRAM access.

Task 2B

Dynamic I-RRIP

Leader sets and decision logic

Sensitivity study

Results

Task 3

Incorporating SPP with Dynamic I-RRIP

Handling prefetched lines

Results